

Dietary Database

By Luis Jaco

For my project, I aimed to create a hospital dietary database. The dietary department in a hospital deals with patient meals. As such, this database aims to contain patient information and aid to create meals which correspond to their given diets. The main complexity of this project is with creating orders which are solely within a patient's diets.

User Roles

Patient

The patient role can do the following:

- View past orders
 - View specific orders contents
- Create orders

Employee

The employee role can do the following:

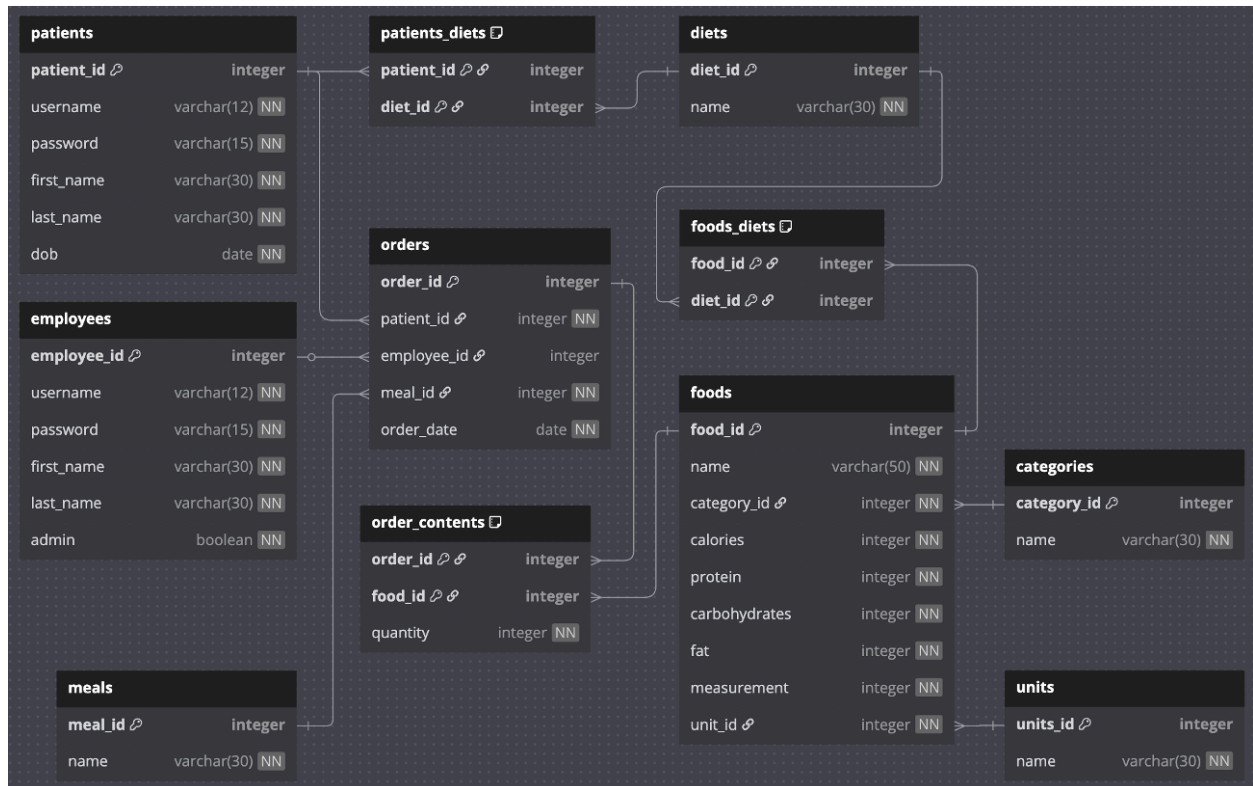
- Select patients
 - View past orders
 - View specific order contents
 - Create orders
 - Edit patient diets
 - Edit past meals

Admin

The admin is very similar to the employee, it can do the following:

- Add patients
- Select patients
 - View past orders
 - View specific order contents
 - Create orders
 - Edit patient diets
 - Edit past meals
 - Change patient password
- Add employees/admins
- Select employees
 - Change employee password

Database Schema



Above is a graphic of the database schema. The tables are as follows:

Employees

Contains employee information (id, username, password, name, admin status).

Patients

Contains patient information (id, username, password, name, date of birth).

Diets

Contains diet information.

Patients_diets

Contains all diets which a patient currently has.

Categories

Contains the names for food categories (appetizer, entree, dessert, cold drinks).

Units

Contains the unit of measurement for a serving of a food (such as ounce, cup, fluid ounce).

Foods

Contains foods along with their serving size and macros (calories, protein, carbohydrates, fat).

Foods_diets

Lists every diet to which a food would be appropriate (such as vanilla ice cream, belongs to a soft, regular, and full liquid diet).

Meals

Contains possible meals for orders (such as breakfast, dinner, lunch)

Orders

Contains order information for every order (meal, patient id, employee id (if employee created order), date).

Orders_contents

Lists every food within a particular order, along with the quantity.

Implementation

Unfortunately, due to the scale and time constraints of this project, I was only able to complete the patient roles functionality.

Sign in

In a typical program scenario, a user can sign into the program by picking between employee or patient sign in, and giving the proper username and password combination.

```
public User signIn(String username, String password, boolean isEmployee) {
    User user = new User();
    try {
        // either use employees or patients table.
        String q = "SELECT * FROM %s WHERE username=? AND password=?".formatted( (isEmployee ? "employees" : "patients"));
        PreparedStatement query = connection.prepareStatement(q);
        query.setString( parameterIndex: 1, username);
        query.setString( parameterIndex: 2, password);

        ResultSet resultSet = query.executeQuery();

        if (resultSet.next()) {
            user = new User(resultSet, isEmployee);
            System.out.println("[!] Sign in successful.");
        }
        if (user.getRank() == 1) {
            getDiets(user);
        }
    } catch (SQLException e) {
        System.out.println("[X] An error occurred.");
    }
    return user;
}
```

```
X-----X
                PLEASE SIGN IN!
[2] Employee sign-in
[1] Patient sign-in
[0] Exit program
X-----X
~%: 1
[!] Enter username:
~%: jjaco
[!] Enter password:
~%: hi_bye
[!] Sign in successful.
```

Select Action

A user will then be able to select their next action.

```
x-----x
                        HOME
[-] PATIENT #2 [jjaco]
jennifer jaco [08-19-2008]

[2] Previous orders
[1] Order meal
[0] Sign out
x-----x
~%: |
```

Previous Orders

Allows a patient to view their orders. We use the following query along with java to produce the results.

```
SELECT
    order_contents.order_id,
    patients.patient_id,
    patients.username AS 'patient',
    employees.username AS 'employee',
    meals.meal_id,
    meals.name AS 'meal',
    order_date,
    SUM(foods.calories) AS 'calories',
    SUM(foods.protein) AS 'protein',
    SUM(foods.carbohydrates) AS 'carbohydrates',
    SUM(foods.fat) AS 'fat'
FROM order_contents
JOIN orders
    ON order_contents.order_id = orders.order_id
JOIN meals
    ON orders.meal_id = meals.meal_id
LEFT JOIN employees
    ON orders.employee_id = employees.employee_id
JOIN patients
    ON orders.patient_id = patients.patient_id
JOIN foods
    ON order_contents.food_id = foods.food_id
GROUP BY order_contents.order_id
HAVING patients.patient_id = ?
ORDER BY order_date DESC, meal_id DESC;
```

```
[+] ORDERS FOR [jennifer jaco]:
. . . . .
[+] ORDER [#6]
patient      | employee | meal      | date      | calories | protein | carbohydrates | fat |
jjaco        | null     | dinner    | 05-01-2025 | 232      | 23g     | 0g            | 15g |
. . . . .
[+] ORDER [#7]
patient      | employee | meal      | date      | calories | protein | carbohydrates | fat |
jjaco        | null     | lunch     | 05-01-2025 | 180      | 22g     | 0g            | 9g  |
. . . . .
```

A patient may then have the option to pick an order and view its specific contents.

```
SELECT
    order_contents.order_id,
    orders.patient_id,
    orders.employee_id,
    orders.meal_id,
    orders.order_date,
    foods.food_id,
    quantity,
    foods.category_id,
    foods.name as 'food',
    foods.calories,
    foods.protein,
    foods.carbohydrates,
    foods.fat,
    foods.measurement,
    units.name as 'unit'
FROM order_contents
JOIN foods
    ON order_contents.food_id = foods.food_id
JOIN units
    ON foods.unit_id = units.unit_id
JOIN orders
    ON order_contents.order_id = orders.order_id
WHERE order_contents.order_id= ? AND patient_id= ?
ORDER BY foods.category_id ASC;
```

```
. . . . .
[+] ORDER [#3]
quantity      | food                                | calories | protein | carbohydrates | fat |
1x [ 1.0 cup ] | chicken noodle soup                | 90       | 6g      | 12g           | 2g  |
1x [ 1.0 cup ] | rice                               | 200      | 4g      | 45g           | 1g  |
1x [ 12.0 fl oz ] | water                               | 0        | 0g      | 0g            | 0g  |
1x [ 1.0 cup ] | garden salad                       | 70       | 2g      | 5g            | 5g  |
1x [ 1.0 cup ] | broccoli                           | 55       | 5g      | 11g           | 1g  |
1x [ 3.0 oz ]  | herb grilled chicken breast        | 165      | 31g     | 0g            | 4g  |
1x [ 0.5 cup ] | fruit cup                           | 60       | 1g      | 15g           | 1g  |
total:         |                                     | 640      | 49g     | 88g           | 14g |
. . . . .
```

Order Meal

To order a meal, a patient must first select the meal it is for (breakfast, lunch, dinner). Then they are freely able to pick categories and select foods to add.

```
X-----X
ORDER MEAL ON [05-01-2025]
[3] Dinner
[2] Lunch
[1] Breakfast
[0] Go back
X-----X
~%: 3
```

```
X-----X
PICK CATEGORY
FOR DIETS: REGULAR
[1] Appetizer | [2] Entree | [3] Vegetable
[4] Bread | [5] Hot drink | [6] Cold drink
[7] Fruit | [8] Dessert | [9] Condiment
[10] Place order
[0] Go back
X-----X
~%: 2
```

Once the category is selected, java will construct a query. This query essentially filters foods based on all diets a patient may have. In the example below, there are two joins to the foods_diets table. Given there was a patient with only one diet (such as the above), the query would only feature one join and WHERE clause.

```
SELECT
  foods.food_id,
  foods.name as 'food',
  foods.category_id,
  measurement,
  units.name as 'unit',
  calories,
  protein,
  carbohydrates,
  fat
FROM foods
JOIN units
  ON foods.unit_id = units.unit_id
JOIN foods_diets fd2
  ON foods.food_id = fd2.food_id
JOIN foods_diets fd5
  ON foods.food_id = fd5.food_id
WHERE category_id = 2
  AND fd2.diet_id=2
  AND fd5.diet_id=5
GROUP BY foods.food_id;
```

```
[!] FOOD OPTIONS:
id | food | portion | calories | protein | carbohydrates | fat |
62 | salmon | 3.0 oz | 232 | 23g | 0g | 15g |
63 | herb grilled chicken breast | 3.0 oz | 165 | 31g | 0g | 4g |
64 | roasted chicken leg | 3.0 oz | 180 | 22g | 0g | 9g |
65 | tuna salad | 0.5 cup | 200 | 20g | 2g | 12g |
66 | hard boiled egg | 1.0 whole | 70 | 6g | 1g | 5g |
67 | scrambled eggs | 0.5 cup | 90 | 6g | 1g | 7g |
68 | rigatoni with sauce | 1.0 cup | 270 | 8g | 42g | 7g |
69 | plain grilled chicken breast | 3.0 oz | 165 | 31g | 0g | 4g |
70 | plain chicken leg | 3.0 oz | 180 | 22g | 0g | 9g |
71 | plain rigatoni | 1.0 cup | 200 | 7g | 42g | 1g |
72 | sunflower butter and jelly sandwich | 1.0 whole | 350 | 9g | 45g | 17g |
73 | turkey burger | 3.0 oz | 200 | 25g | 0g | 10g |
74 | veggie burger | 3.0 oz | 170 | 11g | 20g | 8g |
75 | macaroni and cheese | 1.0 cup | 300 | 12g | 30g | 15g |
[!] Enter food id to add or [0] to go back:
~%: 62
[!] Enter quantity of food [1-10]:
~%: 1
[!] Successfully added food to order.
```

Finally, the user can confirm if they want to create the order, and the order will be sent. These queries will get the id of the next possible order, create an order, and also insert the foods.

```
SELECT MAX(order_id) + 1 AS 'id' FROM orders;
```

```
INSERT INTO orders (order_id, patient_id, employee_id, meal_id, order_date)
VALUES (?, ?, ?, ?, ?);
```

```
String createOrderContents = "INSERT INTO order_contents (order_id, food_id, quantity) VALUES ";
boolean firstValue = true;
for (int foodId : order.getFoods().keySet()) {
  if (firstValue) {
    createOrderContents += "(%d, %d, %d)".formatted(id, foodId, order.getFoodQuantity(foodId));
    firstValue = false;
  } else {
    createOrderContents += ", (%d, %d, %d)".formatted(id, foodId, order.getFoodQuantity(foodId));
  }
}
```

```
INSERT INTO order_contents (order_id, food_id, quantity)
VALUES (?, ?, ?), (?, ?, ?);
```